

AutoTraQ Testing Checklist — GUS (Backend/API)

Login: gus@autotraq.app / autotraq2026 **URL:** https://cs490unco.org **API Base:** https://backend-production-b163.up.railway.app **Role:** Manager

Your section is the **backend API**. You'll test API endpoints directly (using the browser, curl, or Postman) AND verify the frontend correctly communicates with the backend. For each task, mark  or  and note any errors.

0. Setup — Get Your Auth Token

Before testing API endpoints, you need a JWT token:

```
curl -X POST https://backend-production-b163.up.railway.app/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"gus@autotraq.app","password":"autotraq2026"}'
```

Copy the `token` from the response. Use it in all requests below as:

```
-H "Authorization: Bearer YOUR_TOKEN_HERE"
```

1. Health & Server Status (4 tasks)

- ☐ Hit `GET /health` — verify response is `{"status":"ok","database":"connected"}`
- ☐ Check the response **Content-Type** header is `application/json`
- ☐ Verify the **timestamp** in the health response is current (not stale)
- ☐ Hit `GET /api` (base API route) — note what it returns

2. Authentication API (12 tasks)

- ☐ `POST /api/auth/login` with valid credentials — verify you get a token + user object
- ☐ Verify the user object contains: `id`, `email`, `name`, `role`
- ☐ Verify your role is `"manager"` (not `"admin"`)

- [] `POST /api/auth/login` with **wrong password** — verify 401/error response (not 500)
- [] `POST /api/auth/login` with **missing email** — verify validation error
- [] `POST /api/auth/login` with **empty body** — verify it doesn't crash
- [] `GET /api/auth/me` with your token — verify it returns your user info
- [] `GET /api/auth/me` **without** a token — verify 401 Unauthorized
- [] `GET /api/auth/me` with an **invalid/garbage** token — verify 401 (not 500)
- [] `POST /api/auth/change-password` — change your password, then change it back
- [] `POST /api/auth/register` with a new test email — verify account creation works
- [] `GET /api/auth/my-barcode` — verify you get a barcode string back

3. Parts API — CRUD Operations (15 tasks)

- [] `GET /api/parts` — verify it returns a list of parts
- [] Check the response structure: array of objects with `id`, `name`, `sku`, `description`, `condition`, `price`
- [] `GET /api/parts?limit=10` — verify only 10 results return
- [] `GET /api/parts?offset=10&limit=10` — verify you get the second page of results
- [] `GET /api/parts?search=engine` — verify search filtering works
- [] `GET /api/parts?condition=New` — verify condition filtering works
- [] `GET /api/parts/1` — verify a single part returns with full details
- [] `GET /api/parts/999999` — verify 404 response (not 500)
- [] `POST /api/parts` — create a new test part: `json {"name":"Test Part by Gus","sku":"GUS-TEST-001","description":"Testing","condition":"New","price":9.99}` Verify it returns the created part with an `id`
- [] `GET /api/parts/YOUR_NEW_ID` — verify the part you just created exists
- [] `PUT /api/parts/YOUR_NEW_ID` — update the price to 19.99, verify the response
- [] `GET /api/parts/YOUR_NEW_ID` — verify the price actually changed
- [] `DELETE /api/parts/YOUR_NEW_ID` — verify successful deletion
- [] `GET /api/parts/YOUR_NEW_ID` — verify it returns 404 now
- [] Try `POST /api/parts` with **missing required fields** — verify validation error (not 500)

4. Vehicles API (10 tasks)

- [] `GET /api/vehicles` — verify list of vehicles returns
- [] Check response has: `id`, `year`, `make`, `model`
- [] `GET /api/vehicles?limit=5` — verify pagination works
- [] `GET /api/vehicles/makes` — verify list of unique makes returns
- [] `GET /api/vehicles/models` — verify models list returns

- [] GET /api/vehicles/models?make=Toyota — verify filtering by make works
- [] POST /api/vehicles — create a test vehicle: `json {"year":2024,"make":"Tesla","model":"Model 3"}`
- [] GET /api/vehicles/YOUR_NEW_ID — verify it exists
- [] PUT /api/vehicles/YOUR_NEW_ID — change the year to 2025
- [] DELETE /api/vehicles/YOUR_NEW_ID — clean up your test data

5. Inventory API (12 tasks)

- [] GET /api/inventory/locations — verify storage locations list
- [] GET /api/inventory/on-hand — verify on-hand stock data returns
- [] GET /api/inventory/on-hand?partId=1 — verify filtering by part works
- [] GET /api/inventory/events — verify inventory event log
- [] GET /api/inventory/history — verify movement history
- [] GET /api/inventory/top-movers — verify top movers list
- [] GET /api/inventory/dead-stock — verify dead stock identification
- [] POST /api/inventory/locations — create a test location: `json {"name":"Gus Test Bay","code":"GUS-01"}`
- [] POST /api/inventory/receive — receive stock for a part at your new location (check required fields)
- [] POST /api/inventory/correct — make a stock correction
- [] POST /api/inventory/return — process a return
- [] Verify all inventory actions create entries in GET /api/inventory/events

6. Requests API (8 tasks)

- [] GET /api/requests — verify request list
- [] POST /api/requests — create a test request: `json {"partId":1,"quantity":5,"notes":"Gus testing"}`
- [] GET /api/requests/YOUR_NEW_ID — verify the request details
- [] POST /api/requests/YOUR_NEW_ID/approve — approve the request
- [] Verify the status changed to "APPROVED"
- [] POST /api/requests/YOUR_NEW_ID/fulfill — fulfill the request
- [] Verify the status changed to "FULFILLED"
- [] Create another request and test POST /api/requests/ID/cancel

7. SKU & Parts Search API (8 tasks)

- [] GET /api/sku/make-codes — verify make codes return

- [] GET /api/sku/model-codes?make=TOY — verify model codes for a make
- [] GET /api/sku/system-codes — verify system codes
- [] GET /api/sku/component-codes?system=ENG — verify component codes
- [] POST /api/sku/generate — generate a SKU (check required fields)
- [] GET /api/parts-search/advanced?query=engine — verify advanced search
- [] GET /api/parts-search/hierarchy — verify hierarchy tree returns
- [] GET /api/parts-search/hierarchy/ENG/BLK — verify drill-down returns parts

8. CSV API (4 tasks)

- [] GET /api/csv/export — verify a CSV file downloads
- [] Open the CSV — verify it has headers and real data
- [] Check if POST /api/csv/import endpoint exists (don't need to test upload, just verify it's registered)
- [] Try GET /api/csv/export without auth — verify it requires authentication

9. Audit API (4 tasks)

- [] GET /api/audit — verify audit log entries return
- [] Check entries have: timestamp, userId, action, entityType, entityId
- [] GET /api/audit/parts/1 — verify entity-specific audit history
- [] Verify your earlier CRUD operations show up in the audit log

10. Notifications API (4 tasks)

- [] GET /api/notifications — verify notifications list (may be empty)
- [] GET /api/notifications/count — verify unread count returns a number
- [] POST /api/notifications/read-all — verify it succeeds
- [] Verify count is 0 after marking all read

11. Images API (4 tasks)

- [] POST /api/images/primary-bulk with a list of part IDs — check response
- [] GET /api/images/SOME_ID — test fetching an image (if any exist)
- [] GET /api/images/SOME_ID/raw — test raw image endpoint (no auth required)
- [] Verify image endpoints return proper Content-Type headers

12. Error Handling & Edge Cases (10 tasks)

- [] Hit a **non-existent endpoint** (GET /api/fakeroute) — verify clean 404 JSON response
 - [] Send **malformed JSON** in a POST body — verify 400 error (not 500)
 - [] Send a request with an **expired/invalid token** — verify 401
 - [] Try to access an **admin-only endpoint** (e.g., GET /api/auth/users) with your manager token — verify 403 Forbidden
 - [] Test with **very long strings** in request body (1000+ chars for a part name) — does it handle gracefully?
 - [] Test with **negative numbers** (price: -5) — does validation catch it?
 - [] Test with **special characters** in search queries — SQL injection attempt: ' ; DROP TABLE parts; --
 - [] Test **concurrent requests** — fire 10 requests at once, do they all return correctly?
 - [] Verify **CORS** — open browser DevTools Network tab while using the frontend, check no CORS errors
 - [] Check **response times** — are any endpoints >2 seconds? Note which ones.
-

Total: 95 tasks

Tools you can use: - **curl** (command line) - **Postman** (GUI — download free at postman.com) - **Browser DevTools** → Network tab (to see API calls from the frontend)

When done, compile results and send to Anson. Flag any 500 errors or crashes as high priority.